SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM112	**Name:** Rajdeep Mahadev Kamble
Division: B	**Date: 9** October 2026
GitHub: https://github.com/RajDev20	**System:** 8-Puzzle Solver (SLE-2 / SLE-3)

PART A: Architecture Decision Record (ADR)

Title: Use of Breadth-First Search (FIFO Frontier + Explored Set) Instead of Depth-First Search in the 8-Puzzle Solver

Status: Accepted **Date:** 11 October 2026 **PRN & Name:** 25UAM112, Rajdeep Mahadev Kamble

1. Context

My system solves the 8-puzzle (3×3 sliding tiles) and records time, nodes expanded and solution length for each search. The puzzle has 181,440 reachable states, and no heuristic is used in the system, so the choice is between uninformed strategies. My start state is a 25-move random shuffle of the goal, which is solvable and far enough from the goal to show real differences. The solver must return a correct path, and I wanted that choice backed by measured numbers rather than theory alone. In SLE-3 I separated the Puzzle Model from the Search Engine, so BFS and DFS share the same move rules and goal test and differ only in the Frontier type.

2. Decision

I decided to use Breadth-First Search as the solving strategy: the Frontier is a FIFO queue and an Explored Set rejects states already generated. DFS (LIFO stack) is kept in the code only as the baseline for profiling.

3. Alternatives Considered

- **Alternative 1 – DFS (stack + explored set):** implemented and profiled in SLE-2; rejected because of the results below.
- **Alternative 2 – A* / Greedy Best-First with Manhattan distance:** not implemented. It would expand fewer nodes, but my system has no heuristic module (SLE-3 has no A* component), and I wanted an exact, data-backed baseline first.
- **Alternative 3 – Iterative Deepening DFS:** also optimal and low in memory, but it re-expands shallow levels many times; not tested, so I did not choose it without data.

4. Rationale / Justification

The SLE-2 profiling on the identical start/goal state (5 runs) decided this:

Metric (SLE-2)	BFS	DFS	DFS vs BFS
Avg. time (ms)	2.92	57.71	≈ 20× slower
Nodes expanded	2,498	36,813	14.7× more
Solution length (moves)	13 (optimal)	32,669	≈ 2,500× longer
Function calls (cProfile)	22,337	380,743	≈ 17× more

- **Correctness of the answer:** BFS reached the goal in 13 moves, the true shortest path, because it expands the shallowest nodes first. DFS returned a valid but 32,669-move path, which is useless as a puzzle solution.
- **Cost:** BFS stays near O(b^d) with d = 13, while DFS drifted toward O(b^m), where m can reach the 181,440-state space. The measured 14.7× node gap and ≈20× time gap show this clearly.
- **Design fit (SLE-3):** In the Component diagram the Frontier is the only difference (queue vs stack). Choosing BFS meant no change to the Puzzle Model, Explored Set, Goal Test or Path Reconstructor, so the comparison was fair and the decision is easy to reverse.
- **Why not A*:** There is no heuristic in my design yet; BFS gives the optimal reference that a future A* module can be checked against.

5. Consequences

Positive:

- Guaranteed shortest solution (13 moves) and predictable cost on every solvable instance.
- Simple, testable code; the same Search Engine can later host A* by changing only the Frontier to a priority queue.

Negative / Trade-offs:

- BFS is blind: it still expanded 2,498 nodes and stored 4,009 states in the explored set for a 13-move puzzle. Its frontier grows fast with depth (see the scaling table in Part D: ~179k nodes at depth 28, almost the whole 181,440-state space).
- It will not scale to a 15-puzzle (about 1.05×10^{13} states). A heuristic search (A*) would be needed there, which my current system does not include.

6. References

| SLE-2 Profiling Report (BFS vs DFS table) | SLE-3 C4 diagrams (Container and Component levels) | Code: sle4_8puzzle_adr.py

PART B: Short Summary of My SLE Journey

In SLE-1 I built the 8-puzzle search code with an AI Contribution Log and README. In SLE-2 I profiled BFS against DFS on one fixed start state using perf_counter, a node counter, cProfile and py-spy, and BFS won on every metric (2,498 vs 36,813 nodes, 13 vs 32,669 moves). In SLE-3 I drew the full C4 model with six containers and expanded the Search Engine to show that BFS and DFS differ only in the Frontier. In SLE-4 I turned that evidence into one ADR: choosing BFS over DFS. I re-ran the solver for this SLE; it reproduces the same node counts and solution lengths as SLE-2.

PART C: AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** drafting the ADR text, writing the final solver code (sle4_8puzzle_adr.py), and running the extra scaling experiment used in Part D.
- **What I did myself:** chose the decision from my own SLE-2 and SLE-3 work, checked every number against my SLE-2 report, confirmed the code uses my SLE-3 class and function names, ran the script, and prepared my own answers for the viva.